



HCMUS

Viet Nam National University
Ho Chi Minh City
University of Science

fit@hcmus

Seminar: Design Patterns

Topic: Iterator Pattern

Course: CS202- Programming Systems

Supervisor:

PhD. DINH Ba Tien

MSc. TRUONG Phuoc Loc

MSc. HO Tuan Thanh

Group 03

Full Name	Student ID
Đỗ Viết Hoàng Long	25125024
Trần Như Khải	25125045
Trần Đăng Khoa	25125056
Trần Minh Khoa	25125057

Contents

1	Introduction	2
1.1	Overview & Context	2
1.2	Motivation	2
1.3	Objectives and Scope	2
1.4	Report Organization	2
2	Iterator Pattern	3
2.1	Problem Statement	3
2.1.1	Problem Context	3
2.2	Proposed Solution	3
2.2.1	Solution Architecture	3
2.2.2	C++ Implementation Example	4
2.3	Evaluation	5
2.3.1	Advantages and Disadvantages	5
2.3.2	Iterator Invalidation and Object Lifetime Pitfalls	6
2.3.3	High-Performance Optimization Techniques in C++	6
2.3.4	Concurrency, Mutation Handling, and C++ Standards	7
2.4	Applications	8
2.4.1	Relationship with Other Design Patterns	8
2.5	Quiz	10
3	References	12

1 Introduction

1.1 Overview & Context

In modern software engineering, the ability to design systems that are flexible, maintainable, and scalable is of paramount importance. Design patterns serve as standardized, reusable solutions to common problems in Object-Oriented Design (OOD), enabling developers to build robust and adaptable architectures [1]. Among the various categories of design patterns, behavioral patterns specifically address algorithms and the assignment of responsibilities between objects, improving communication flow and encapsulation within a system.

1.2 Motivation

A recurring challenge in application development is the need to access and traverse elements of an aggregate object without exposing its underlying internal structure (such as arrays, linked lists, trees, or hash tables) [1, 2]. Exposing low-level traversal logic to client code introduces tight coupling, violates encapsulation, and breaks fundamental SOLID design principles. The **Iterator Pattern** [1] provides an elegant solution by extracting navigation state and traversal algorithms into dedicated iterator objects, enabling uniform and decoupled data access across diverse collection types.

1.3 Objectives and Scope

This report aims to deliver a comprehensive and critical analysis of the Iterator pattern within the context of the CS202 course. The main objectives are as follows:

- To formulate and analyze the underlying architectural problems motivating the Iterator pattern.
- To examine the structural components, interface contracts, and implementation mechanics of iterators.
- To evaluate performance trade-offs, modern C++ optimization techniques (such as zero-cost stack-allocated iterators [3, 4] and internal iterators), and concurrency handling strategies [5, 6].
- To explore real-world applications and compare the Iterator pattern with related behavioral patterns (Visitor, Chain of Responsibility) [1].

1.4 Report Organization

The remainder of this report is organized as follows:

- **Section 2:** Presents a detailed analysis of the Iterator pattern, including problem statement, proposed solution architecture, evaluation and C++ optimization techniques, real-world applications, and quiz questions.
- **Section 3:** Lists the references and supporting literature used in this report.

2 Iterator Pattern

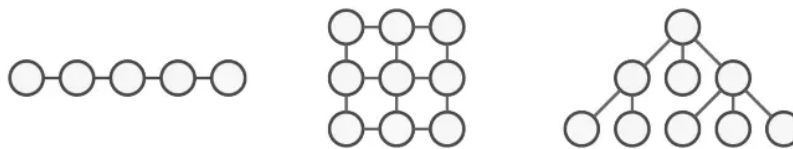
2.1 Problem Statement

2.1.1 Problem Context

In software architecture, applications often manage elements stored across multiple disparate data structures. For example, a modern media streaming application may maintain an array-based queue for upcoming tracks, a doubly linked list for play history, and a binary search tree (BST) for genre organization.

Without an abstraction layer, the client application must implement specific traversal logic tailored to each container:

- **Array / Contiguous Vector:** Relies on direct indexed access (`list[i]`).
- **Linked List:** Relies on node-by-node pointer traversal (`node->next` or `node->prev`).
- **Tree / Graph:** Requires recursive or stack-based traversal strategies (e.g., in-order, pre-order, DFS, or BFS).



Various types of collections.

Figure 2.1: Problem of Direct Container Traversal [7]

Exposing these low-level traversal mechanisms directly to consumer code introduces several architectural deficiencies and violates core object-oriented design principles [1, 2]:

- **Encapsulation Leakage:** The internal structure, node pointers, and indexing models of the collection are exposed directly to the client.
- **Single Responsibility Principle (SRP) Violation:** The aggregate class is burdened with managing traversal state and access algorithms in addition to managing its core dataset.
- **Open/Closed Principle (OCP) Violation:** Introducing a new collection type or altering an existing data structure requires updating all client-side traversal loops across the codebase.
- **Code Duplication:** Traversal and rendering algorithms must be rewritten or overloaded for every individual container type.

2.2 Proposed Solution

2.2.1 Solution Architecture

The **Iterator Pattern** [1] decouples traversal algorithms from collection data structures by extracting navigation state into an external, dedicated object known as an **Iterator**.

The pattern establishes two primary interface hierarchies [1, 2]:

1. **Aggregate Interface (Iterable):** Defines a factory method (e.g., `createIterator()`) that supplies a compatible traversal cursor without revealing the internal container layout.
2. **Iterator Interface:** Exposes a standard traversal contract—traditionally `hasNext()` and `next()`, or `first()`, `next()`, `isDone()`, and `currentItem()`.

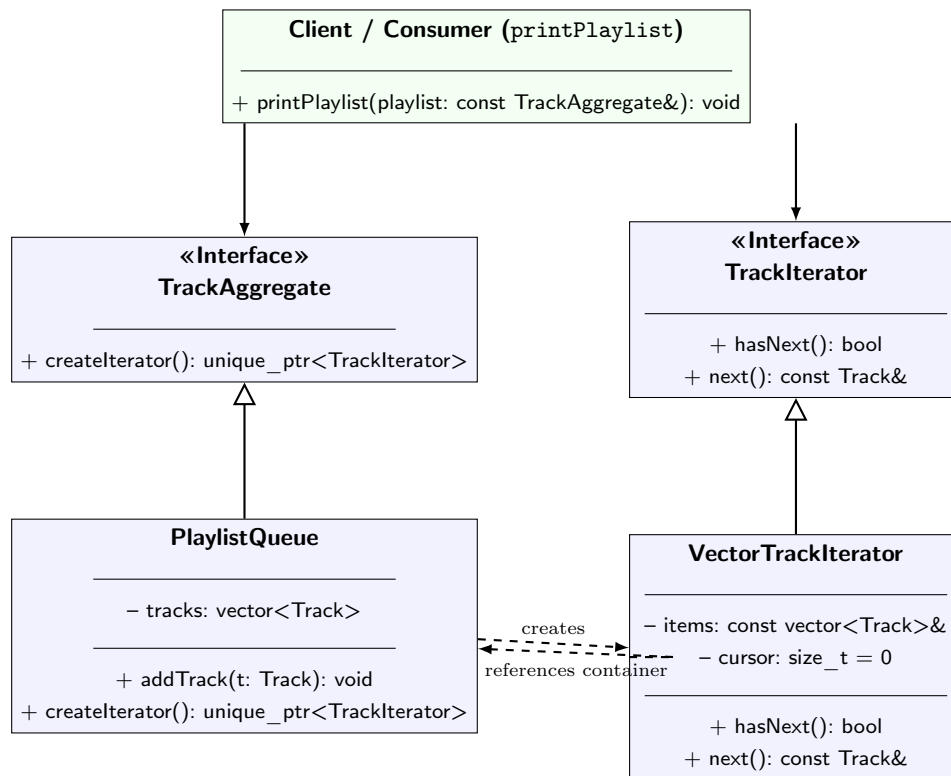


Figure 2.2: UML Class Diagram for Concrete Media Playlist Iterator Implementation (Adapted from [7])

2.2.2 C++ Implementation Example

Below is the robust C++ implementation demonstrating the aggregate queue decoupled from its consumer via an iterator interface. Notice the defensive boundary checking inside `next()` to eliminate undefined behavior:

```

1 #include <iostream>
2 #include <memory>
3 #include <vector>
4 #include <string>
5 #include <stdexcept>
6
7 // Target data element
8 struct Track {
9     std::string title;
10    std::string artist;
11 };
12
13 // 1. Iterator Interface
14 class TrackIterator {
15 public:
16     virtual ~TrackIterator() = default;
17     virtual bool hasNext() const = 0;
18     virtual const Track& next() = 0;
19 };
20
21 // 2. Aggregate Interface
22 class TrackAggregate {
23 public:
24     virtual ~TrackAggregate() = default;
25     virtual std::unique_ptr<TrackIterator> createIterator() const = 0;
26 };
27
28 // 3. Concrete Aggregate
29 class PlaylistQueue : public TrackAggregate {
30 private:
31     std::vector<Track> tracks;
32
33     // Concrete Iterator as an inner/friend class
  
```

```

34 class VectorTrackIterator : public TrackIterator {
35 private:
36     const std::vector<Track>& items;
37     size_t cursor = 0;
38
39 public:
40     explicit VectorTrackIterator(const std::vector<Track>& target)
41         : items(target) {}
42
43     bool hasNext() const override {
44         return cursor < items.size();
45     }
46
47     const Track& next() override {
48         if (!hasNext()) {
49             throw std::out_of_range("VectorTrackIterator::next(): Iterator out of
50 bounds.");
51         }
52         return items[cursor++];
53     }
54 };
55
56 public:
57     void addTrack(const Track& track) {
58         tracks.push_back(track);
59     }
60
61     std::unique_ptr<TrackIterator> createIterator() const override {
62         return std::make_unique<VectorTrackIterator>(tracks);
63     }
64 };
65
66 // 4. Client Usage (Uniform Traversal)
67 void printPlaylist(const TrackAggregate& playlist) {
68     std::unique_ptr<TrackIterator> it = playlist.createIterator();
69     int index = 1;
70     while (it->hasNext()) {
71         const Track& t = it->next();
72         std::cout << index++ << ". " << t.title
73             << " by " << t.artist << "\n";
74     }
75 }

```

Listing 1: C++ Implementation of Playlist Iterator Pattern

2.3 Evaluation

2.3.1 Advantages and Disadvantages

Advantages

- **Strict Encapsulation:** Client code never accesses the internal node pointers or collection arrays directly [1].
- **Single Responsibility Principle:** Traversal logic and cursor bookkeeping are isolated inside the iterator class, keeping the container focused solely on data storage.
- **Open/Closed Principle:** New data structures or alternative traversal strategies (e.g., filtered, reverse, or breadth-first) can be added without modifying existing consumer loops.
- **Independent Concurrent Traversals:** Because each iterator maintains its own cursor state, multiple independent traversals can run across the same underlying container simultaneously.
- **Lazy Evaluation / Memory Efficiency:** Elements can be processed on-demand, enabling safe traversal of streaming data, infinite sequences, or large database cursors without exhausting memory.

Disadvantages

- **Over-Engineering for Simple Needs:** Implementing polymorphic interfaces, factory methods, and custom iterator classes introduces unnecessary complexity for simple, fixed-size structures.

- **Indirection Overhead:** Virtual function dispatch and dynamic memory allocations (`new` / `std::make_unique`) introduce CPU and heap overhead compared to native index loops [3].

2.3.2 Iterator Invalidation and Object Lifetime Pitfalls

In native C++, memory safety and object lifetimes are critical considerations when implementing custom iterators [3, 4]:

- **Dangling References / Lifetime Discrepancy:** A concrete iterator typically retains a raw reference (e.g., `const std::vector<Track>&`) or a pointer to the aggregate container. If the aggregate object's lifetime ends before the iterator is destroyed (for example, if the aggregate was created as a temporary rvalue and discarded), subsequent calls to `hasNext()` or `next()` will dereference dangling memory, resulting in undefined behavior.
- **Structural Mutation during Traversal:** If the underlying container is structurally modified (e.g., inserting or appending elements via `push_back()`) while an iterator is actively traversing it, `std::vector` may trigger internal heap reallocation [3]. This moves the underlying buffer to a new memory address, invalidating any active pointers or indices held by external iterators.
- **Architectural Mitigations:** To safeguard against invalidation in production systems, engineers can: (1) enforce strict scoped iteration where container lifetimes strictly enclose iterator lifetimes, (2) share ownership via `std::shared_ptr<const Container>`, or (3) adopt snapshot-based iterators.

2.3.3 High-Performance Optimization Techniques in C++

Zero-Cost Abstractions and Stack-Allocated Iterators In performance-critical C++, virtual method calls and heap allocation can be bypassed entirely by utilizing stack-allocated value types that satisfy the C++ standard range idiom (`begin()` and `end()`) [3, 4]:

```

1 class HighPerformanceBuffer {
2 private:
3     std::vector<Track> buffer;
4
5 public:
6     // Lightweight Stack-Allocated Iterator
7     struct Iterator {
8         const Track* ptr;
9
10        bool operator!=(const Iterator& other) const {
11            return ptr != other.ptr;
12        }
13        const Track& operator*() const { return *ptr; }
14        Iterator& operator++() { ++ptr; return *this; }
15    };
16
17    Iterator begin() const { return Iterator{buffer.data()}; }
18    Iterator end() const {
19        return Iterator{buffer.data() + buffer.size()};
20    }
21 };
22
23 // Enables compiler-inlined, zero-cost range-based for loops:
24 void renderAll(const HighPerformanceBuffer& buff) {
25     for (const auto& track : buff) {
26         std::cout << track.title << "\n";
27     }
28 }

```

Listing 2: Zero-Cost Stack-Allocated Iterator in C++

Internal Iterators via Higher-Order Functions Unlike external iterators where the client drives traversal manually via loops, internal iterators encapsulate loop execution inside the aggregate and apply a callback to each item:

```

1 class SongCollection {
2     std::vector<Track> songs;
3 public:
4     template <typename Callback>

```

```

5 void forEach(Callback action) const {
6     for (const auto& song : songs) {
7         action(song);
8     }
9 }
10 };

```

Listing 3: Internal Iterator via Functional Callback

2.3.4 Concurrency, Mutation Handling, and C++ Standards

Modifying a collection while an iterator is actively traversing it leads to race conditions, invalid iterators, or data races [5]. Table 2.1 contrasts standard synchronization techniques:

Table 2.1: Comparison of Thread-Safe Iteration Strategies

Technique	Mechanism	Advantages	Disadvantages
Shared Mutex (<code>std::shared_mutex</code>)	Uses shared locks for reading/iteration; exclusive locks for writing [5].	Simple; minimal memory overhead.	High read load can starve pending write operations.
Defensive Snapshot	Iterator duplicates the underlying collection at creation time.	Eliminates reader-writer conflicts completely.	Allocation and copying overhead proportional to collection size.
Atomic Swap (Copy-on-Write)	Traversals read an immutable shared snapshot; writes clone and atomically swap pointers [5, 6].	Non-blocking read traversal; reader threads never block on writer threads.	Lock-free capability is implementation-defined; write-amplification cost.

```

1 #include <iostream>
2 #include <vector>
3 #include <memory>
4 #include <atomic>
5
6 // Note: std::atomic<std::shared_ptr<T>> is standardized in C++20 (ISO/IEC 14882:2020).
7 // In C++11/C++14/C++17, the equivalent was std::atomic_load/store free functions.
8 class LockFreeTrackCatalog {
9 private:
10     std::atomic<std::shared_ptr<const std::vector<Track>>> catalog_ptr;
11
12 public:
13     LockFreeTrackCatalog() {
14         catalog_ptr.store(std::make_shared<const std::vector<Track>>());
15     }
16
17     // Read traversal: readers acquire an immutable snapshot without locking mutexes.
18     void traverse() const {
19         std::shared_ptr<const std::vector<Track>> snapshot = catalog_ptr.load();
20         for (const auto& track : *snapshot) {
21             std::cout << track.title << "\n";
22         }
23     }
24
25     // Write operation using atomic compare-and-swap (CAS)
26     void addTrack(const Track& track) {
27         std::shared_ptr<const std::vector<Track>> old_data;
28         std::shared_ptr<const std::vector<Track>> new_data;
29
30         do {
31             old_data = catalog_ptr.load();
32             auto copy = std::make_shared<std::vector<Track>>>(*old_data);
33             copy->push_back(track);
34             new_data = copy;
35         } while (!catalog_ptr.compare_exchange_weak(old_data, new_data));
36     }
37 }

```

```

38 // Utility to verify platform lock-free guarantee at runtime
39 bool checkLockFree() const {
40     return catalog_ptr.is_lock_free();
41 }
42 };

```

Listing 4: Copy-on-Write Snapshot Traversal in C++20

Important ISO C++ Standard Note (Lock-Freedom): According to the C++ Working Draft [6, 5], whether `std::atomic<std::shared_ptr<T>` is strictly lock-free on a given platform is **implementation-defined**. It relies on hardware support for double-word compare-and-swap (DWCAS / CMPXCHG16B) or small internal table-based locks. Developers can check `catalog_ptr.is_lock_free()` at runtime or query `std::atomic<std::shared_ptr<T>::is_always_lock_free` at compile time.

2.4 Applications

The Iterator pattern is extensively utilized across modern software engineering and systems programming to provide unified, decoupled traversal across heterogeneous data structures and streaming resources [1, 2]:

- **Hierarchical DOM Traversal:** Modern web rendering engines use iterators to traverse recursive DOM trees without exposing internal node linkage.
- **Infinite Scrolling and Pagination:** Mobile and web feeds leverage iterators to paginate remote API data on demand, keeping RAM usage low.
- **Offline Sync Queues:** Mobile systems iterate sequentially through persistent task logs to replay operations after reconnecting to a network.

2.4.1 Relationship with Other Design Patterns

Table 2.2: Architectural Comparison: Iterator vs. Visitor

Dimension	Iterator Pattern [1]	Visitor Pattern [1]
Core Objective	Access and sequence traversal without exposing container internals.	Adding new operations to class hierarchies without modifying their source code.
Target Elements	Primarily homogeneous or structured collections.	Heterogeneous object structures and composite hierarchies.
Dispatch Model	Single dispatch (<code>it->next()</code>).	Double dispatch (<code>element->accept(visitor)</code>).
Traversal Control	Explicitly driven by the Iterator cursor and client consumption loop.	Unconstrained by the pattern; traversal can be delegated to the Object Structure, an Iterator, or the Visitor itself.

Iterator vs. Visitor Synergy: An Iterator can traverse a complex composite hierarchy sequentially, passing each node to a Visitor that executes polymorphic operations without altering the tree structure [1].

Iterator vs. Chain of Responsibility Synergy (Processing Pipelines): The two patterns frequently collaborate in data processing pipelines, middleware stacks, and stream filtration [2]:

- The **Iterator** acts as the *data source pump*, fetching items one-by-one from complex, memory-efficient data structures without exposing internal node layouts.
- The **Chain of Responsibility** acts as the *processing pipeline*, routing each item produced by the iterator through a series of discrete, loosely coupled validation, filtering, or transformation stages.

Table 2.3: Architectural Comparison: Iterator vs. Chain of Responsibility

Dimension	Iterator Pattern [1]	Chain of Responsibility Pattern [1]
Core Objective	Traverses elements sequentially across a collection without exposing its internal structure.	Passes a request along a dynamic chain of potential handlers until one processes it.
Flow Control	Sequential and deterministic; visits every element in order unless explicitly terminated by the client.	Conditional and delegated; execution passes down the handler link until a condition or termination criteria is met.
Decoupling Target	Decouples the consumer algorithm from the underlying data structure layout.	Decouples the sender of a request from its potential receivers/handlers.
State Management	Tracks iteration state (cursors, indices, tree pointers) explicitly.	Handlers maintain references only to their immediate successor in the chain.

```

1 #include <iostream>
2 #include <memory>
3 #include <string>
4 #include <vector>
5
6 struct Track {
7     std::string title;
8     int durationSec;
9     bool isExplicit;
10 };
11
12 // --- CHAIN OF RESPONSIBILITY: Handler Interface ---
13 class TrackFilter {
14 protected:
15     std::shared_ptr<TrackFilter> nextHandler;
16
17 public:
18     virtual ~TrackFilter() = default;
19
20     void setNext(std::shared_ptr<TrackFilter> next) {
21         nextHandler = next;
22     }
23
24     virtual bool process(const Track& track) {
25         if (nextHandler) {
26             return nextHandler->process(track);
27         }
28         return true; // Passed all pipeline stages
29     }
30 };
31
32 // Concrete Handler 1: Filter explicit content
33 class ExplicitFilter : public TrackFilter {
34 public:
35     bool process(const Track& track) override {
36         if (track.isExplicit) {
37             std::cout << "[Blocked - Explicit]: " << track.title << "\n";
38             return false;
39         }
40         return TrackFilter::process(track);
41     }
42 };
43
44 // Concrete Handler 2: Filter out short tracks (< 30 seconds)
45 class DurationFilter : public TrackFilter {
46 public:
47     bool process(const Track& track) override {

```

```
48     if (track.durationSec < 30) {
49         std::cout << "[Skipped - Too Short]: " << track.title << "\n";
50         return false;
51     }
52     return TrackFilter::process(track);
53 }
54 };
55
56 // --- ITERATOR + CHAIN OF RESPONSIBILITY IN ACTION ---
57 void runAudioPipeline(TrackIterator& it, TrackFilter& pipeline) {
58     std::cout << "--- Processing Tracks Through Filter Pipeline ---\n";
59     while (it.hasNext()) {
60         const Track& track = it.next();
61         // Iterator provides the item; Chain processes/filters the item
62         if (pipeline.process(track)) {
63             std::cout << "[Playing]: " << track.title << "\n";
64         }
65     }
66 }
```

Listing 5: Synergy between Iterator and Chain of Responsibility

2.5 Quiz

To evaluate understanding of the Iterator pattern, consider the following technical scenarios:

Question 1: Architectural Intent and Design Principles

Scenario: Which software design principle is most directly preserved when traversal state and iteration algorithms are moved out of the container class and into a separate Iterator class?

- (A) Liskov Substitution Principle (LSP).
- (B) Single Responsibility Principle (SRP).
- (C) Interface Segregation Principle (ISP).
- (D) Dependency Inversion Principle (DIP).

Answer: (B). By delegating traversal mechanics and cursor bookkeeping to the Iterator, the aggregate class focuses exclusively on managing its data elements, satisfying the Single Responsibility Principle.

Question 2: Concurrency and Thread Safety

Scenario: A developer needs to traverse a collection from multiple reader threads while a background thread occasionally appends new elements. If avoiding reader lock contention and achieving maximum read performance are the highest priorities, which strategy is most effective?

- (A) Using a single global mutex locking both reads and writes during every `next()` call.
- (B) Standard pointer-based iteration with volatile keywords.
- (C) Copy-on-Write (Atomic Pointer Swap) with immutable snapshot traversal.
- (D) Removing iterators and exposing the underlying array index.

Answer: (C). Copy-on-Write with atomic pointer swapping allows readers to iterate over an immutable snapshot without acquiring mutex locks, completely eliminating reader lock contention.

Question 3: C++ Optimization and Idioms

Scenario: In modern performance-critical C++, why are stack-allocated iterators with overloaded `operator++`, `operator*`, and `operator!=` often preferred over classic polymorphic iterators implementing pure virtual methods?

- (A) Polymorphic iterators do not support C++ syntax.

- (B) Stack-allocated range iterators avoid virtual dispatch overhead and dynamic heap allocations, enabling aggressive compiler inlining and zero-cost abstractions.
- (C) Virtual methods cannot be used inside loops in C++.
- (D) Polymorphic iterators consume less memory than pointer iterators.

Answer: (B). Stack-allocated iterators adhering to C++ range concepts eliminate the runtime overhead of vtable lookups and heap allocations (`new/std::make_unique`), permitting the compiler to optimize the loop down to raw pointer arithmetic.

Question 4: Memory Safety and Iterator Invalidation

Scenario: In C++, while an iterator is actively traversing a `std::vector`, a concurrent thread or a callback modifies the container by calling `push_back()`. Why does this operation carry a high risk of undefined behavior?

- (A) `push_back()` triggers dynamic heap reallocation if capacity is exceeded, moving elements to a new memory address and leaving the iterator with dangling pointers.
- (B) `std::vector` immediately throws a compile-time assertion error.
- (C) The iterator automatically detects the mutation and enters a deadlock.
- (D) The vector elements are reversed, causing infinite loops.

Answer: (A). Appending elements can exceed the current capacity of `std::vector`, prompting memory reallocation. This moves the underlying buffer and invalidates all existing pointers, references, and iterators pointing into the old storage.

Question 5: Pattern Synergy (Iterator and Visitor)

Scenario: You are designing a complex document rendering engine that processes a hierarchical composite tree of nodes (e.g., text, images, tables). You need to traverse all elements sequentially and execute distinct export operations (HTML, PDF, Markdown) without modifying the node classes. What is the most cohesive architectural combination?

- (A) Use only the Singleton pattern for the whole document tree.
- (B) Use an Iterator to handle sequential tree navigation and decouple structure traversal, passing each visited node to a Visitor that executes double-dispatch polymorphic operations.
- (C) Implement concrete export logic inside each node class using hardcoded `switch-case` statements.
- (D) Flatten the tree into an array and use only the Chain of Responsibility pattern.

Answer: (B). The Iterator pattern encapsulates traversal state across the composite hierarchy, while the Visitor pattern encapsulates extensible polymorphic operations across heterogeneous node types via double dispatch, providing a clean and maintainable synergy.

3 References

- [1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [2] Alexander Shvets. Iterator design pattern, 2023. Accessed: Aug 2026.
- [3] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Professional, 4th edition, 2013.
- [4] Scott Meyers. *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media, 2014.
- [5] Anthony Williams. *C++ Concurrency in Action*. Manning Publications, 2nd edition, 2019.
- [6] ISO/IEC JTC 1/SC 22/WG 21. Iso/iec 14882:2020: Programming languages — c++, 2020.
- [7] Viblo Community. Iterator design pattern: Trợ thủ đắc lực của developers, 2021. Accessed: Aug 2026.